STUDY NOTES

AI Consultant Track

Every module on one page each, with diagrams. Each page ends with a line to say out loud in an interview.

1. LLM Foundations & Inference Economics
2. Classical ML & Decision Science
3. Data Foundations & Readiness
4. Embeddings, Retrieval & Vector Search
5. RAG Architecture
6. Agents & Orchestration
7. Evaluation & Quality Systems
8. Security, Privacy & Adversarial Robustness
9. Deployment & Production Engineering
10. Economics, TCO, Vendors & Procurement
11. Governance, Risk & Responsible AI
12. Consulting Craft & the Case Method
13. General Case Craft
14. Multimodal, Voice & Arabic AI
15. Interview playbook (Frameworks)

LLM Foundations & Inference Economics

1. The core loop

An LLM is a deep stack of identical blocks. Each block does two things: **self-attention** (every token looks back at the earlier tokens and decides which matter) and a **feed-forward** step (each token is transformed on its own). Stack 60 to 100 of these, train on trillions of tokens to predict the next one, and you have a frontier model.

- The decisive fact: it writes **one token at a time, left to right**, feeding each new token back in
- That loop explains almost everything operational: why output costs more, why long answers are slow, why caching works, why chain-of-thought helps

2. Attention

- Each token computes a **Query** (what am I looking for?), a **Key** (what do I offer?) and a **Value** (what do I contribute if picked)
- Its Query is scored against every earlier Key, the scores become weights that sum to 1, and the output is the weighted sum of Values
- **Quadratic cost**: every token attends to every earlier token, so doubling the context roughly quadruples attention compute

> ⚠ Don't stuff everything into context. 4k relevant tokens from retrieval usually beat a 100k-token dump on both quality and cost.

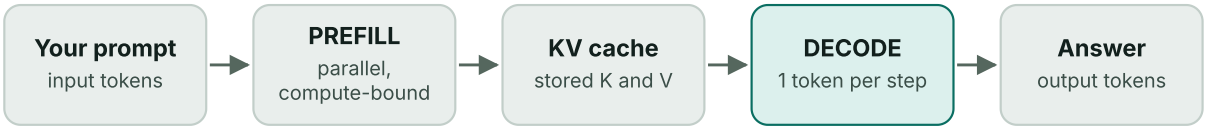
3. Four upgrades to name

- **GQA** (grouped-query attention): groups of query heads share one Key/Value pair, shrinking the KV cache 4 to 8× with little quality loss
- **MoE** (mixture of experts): many expert sub-networks, a router activates only a few per token. **Active** parameters drive cost; **total** parameters drive memory. Never compare models by headline parameter count
- **RoPE**: how word position is injected. Long-context versions are retrained, not switched on. **Lost-in-the-middle**: facts buried mid-context are recalled less reliably, so put critical material at the start or end
- **Reasoning models**: generate hidden thinking tokens before answering. Better on hard problems, but thinking is billed as output and adds latency. Telling them 'think step by step' is redundant

4. The two phases of generation

- **Prefill** (reading your input): one parallel pass that builds the KV cache. Compute-bound, cheap per token, and it sets **TTFT** (time to first token, ~0.5 to 2 s)
- **Decode** (writing the output): one token per step, each step reading the whole KV cache. Memory-bandwidth-bound, sets **TPOT** (~15 to 40 ms per token)
- That asymmetry is why **output costs ~5× input** (e.g. $3 in / $15 out per million tokens)
- KV cache size per token = 2 × layers × KV heads × head dimension × bytes per value. A modern 70B GQA model is about 0.33 MB per token, so ~33 GB at 100k context

✎ DIAGRAM



Prefill is cheap and sets time-to-first-token; decode is sequential, reads the whole KV cache every step, and costs about 5× per token.

5. Tokens and the multilingual tax

- A token is a sub-word chunk from **byte-pair encoding**: roughly 4 characters or 0.75 English words
- Tokenizers are trained mostly on English, so other scripts need more tokens for the same meaning: **Arabic ~1.5 to 2.5×**, French ~1.2 to 1.6×
- Triple hit: higher cost, higher latency (more decode steps) and less effective context
- The professional move: a 10-minute benchmark of tokens per 100 characters on real target-language text, turned into a **per-language budget multiplier**

6. Sampling and the determinism myth

- **Temperature** sharpens or flattens the next-token distribution: 0 to 0.3 for extraction and structured output, 0.5 to 0.8 for chat and RAG, ~1.0 for creative work
- **top_p** keeps only the most likely tokens up to a probability mass p (0.9 to 0.95). Tune temperature or top_p, not both
- Temperature 0 is **not** deterministic: GPU floating-point maths, request batching, MoE routing and provider hardware all vary

- For regulated clients promise **reproducibility of process**: pinned model versions, logged prompts and parameters, archived outputs

7. Why models fail

- **Hallucination**: a confident, fluent, false statement. It's a consequence of predicting plausible text, not true text. **Intrinsic** = contradicts the source you gave it (fix with grounding and faithfulness evals); **extrinsic** = unverifiable from any source (fix with retrieval)
- **Miscalibration**: the model sounds equally sure whether right or wrong, so don't trust its stated confidence
- **Sycophancy**: it agrees with the user because training rewarded agreeable answers
- Hard limits without tools: exact arithmetic, strict counting, long-horizon planning

8. Structured output

- Reliability ladder, best first: **constrained decoding / strict schema** → forced tool call → JSON mode → 'please reply in JSON' (fails 1 to 5% of the time forever)
- Validate every payload on receipt anyway; it turns silent corruption into a loud error

9. The four kinds of caching

- **KV cache**: inside one request, automatic. Not a fix you propose
- **Prompt (prefix) caching**: a provider feature across requests. Reuses the processing of an **identical prompt start**; cached tokens are ~90% cheaper and the first token comes sooner. The model still writes a fresh answer. Put stable content first, variable content last
- **Response cache**: your system stores the full answer for an exact repeat question and skips the model
- **Semantic cache**: returns a stored answer for a *similar* question. Biggest saving, biggest risk; only for stable FAQs with a strict threshold, never for personal or live data

> ⚠ One timestamp or user name at the top of the prompt silently kills the prompt cache.

✎ DIAGRAM

Cache	Lives where	Saves	Risk
KV cache	Inside one call	Recomputing past tokens	None: how inference works
Prompt cache	Provider, across calls	Re-reading a fixed prefix	Misses if the prefix changes
Response cache	Your system	Whole call, exact repeat	Stale answers
Semantic cache	Your system	Whole call, similar question	Wrong answer to a near-miss

Four different things called 'caching'. Mixing them up is an instant credibility leak.

10. Prompt engineering toolkit

- **Zero-shot** with clear structure first; add **few-shot** examples (2 to 5, chosen to cover tricky cases) only if needed
- Say who the output is for and what it's used for; separate instructions, context and the question
- **Chain-of-thought** for non-reasoning models only; **self-consistency** (sample several, take the majority) for accuracy at extra cost
- **Prompt chaining**: split big tasks into small testable calls (extract → classify → draft)
- Version prompts in git, test every change against the golden set, change one thing at a time

11. Cost levers, in order

1. **Cut output tokens** (output is a small share of tokens but close to half the bill)
2. **Cache the stable prefix**
3. **Route by difficulty**: cheap model for easy work, strong model for hard work
4. **Batch API** for anything not interactive (~50% off)

- Prices change monthly: quote only from the provider's live page with an 'as of' date

SAY IT

"Output costs about 5× input because every output token is its own pass over the whole KV cache, so I'd cut output first, then cache the stable prefix, then route easy requests to a cheaper model."

Classical ML & Decision Science

1. Why this module matters

A large share of enterprise value (churn, demand forecasting, fraud, recommendations, risk scoring) is not an LLM problem. The most senior move in many engagements is talking the client **out of** an LLM.

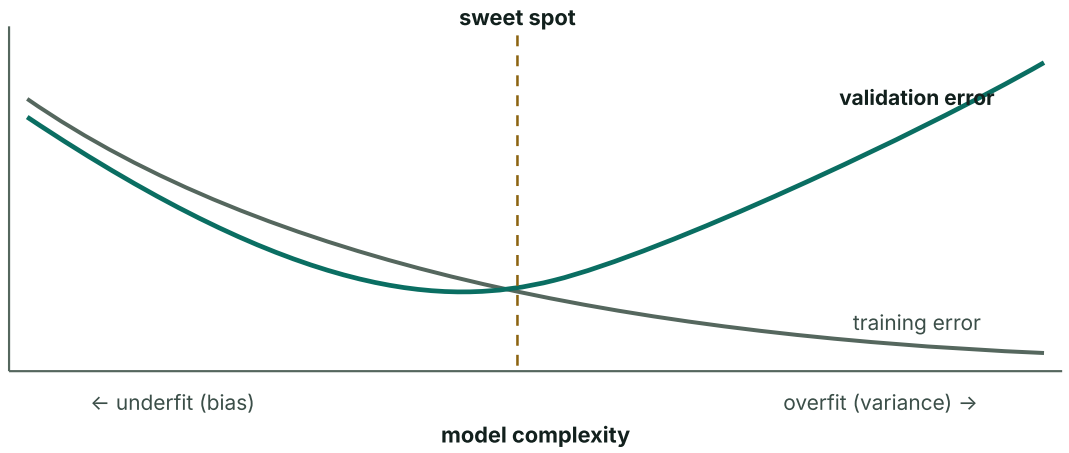
2. Vocabulary

- **Features** = the input variables · **Label / target** = what you predict
- **Training** = fitting the model to labelled history · **Inference / scoring** = applying it to new cases
- **Supervised** = you have labelled outcomes · **Unsupervised** = no labels, you look for structure (e.g. customer segments)
- **Classification** predicts a category (churn or not); **regression** predicts a number (next month's revenue)
- Most classifiers output a probability; the **threshold** that turns it into a yes/no is a business decision about the cost of false alarms vs missed cases

3. Overfitting and honest testing

- Too simple → **underfits** (high bias). Too complex → memorises noise and **overfits** (high variance): perfect on training data, poor on new data
- Spot overfitting as a big gap between training and validation scores
- Split data into **train / validate / test**, keep the test set untouched until the end, and use cross-validation to tune

✎ DIAGRAM



Training error keeps falling as the model gets more complex; validation error falls then rises. The sweet spot is the bottom of the validation curve.

4. Metrics that matter

- **Accuracy trap**: a fraud model that always says 'legitimate' scores 99.9% and catches nothing
- **Precision**: of everything I flagged, how much was really positive? (are the alarms trustworthy?)
- **Recall**: of all the real positives, how many did I catch?
- **F1** balances the two · **AUC** measures separation across all thresholds · **MAE / RMSE** for regression (RMSE punishes big misses more)
- The consultant's job: translate precision vs recall into money (cost of a false alarm vs cost of a missed case)

✎ DIAGRAM

	Model says YES	Model says NO
Really YES	True positive	False negative (missed)
Really NO	False positive (false alarm)	True negative

Precision = TP ÷ (TP + FP). Recall = TP ÷ (TP + FN).

5. Model families

- **Logistic / linear regression**: the interpretable baseline you never skip
- **Gradient-boosted trees** (XGBoost, LightGBM, CatBoost): trees built in sequence, each fixing the last one's errors. **Still state of the art on tabular business data**, fast and explainable

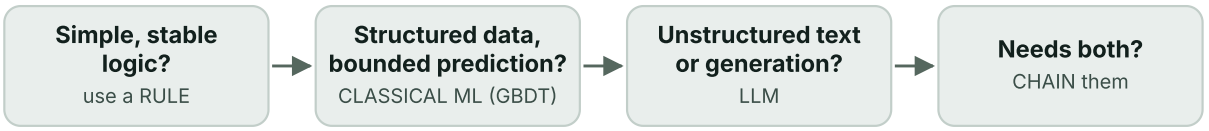
- **Random forest**: robust, low-tuning default · **k-means**: unsupervised segmentation
- **Neural nets**: images, audio, text. Usually the wrong, expensive choice for a spreadsheet

6. Causation, A/B tests and uplift

- A predictive model finds **correlations**: enough to forecast, never enough to justify an intervention (loyal customers using feature X doesn't mean X causes loyalty)
- **A/B test**: randomly split into treatment and control; because assignment is random, any difference is caused by the change. Pitfalls: no pre-set metric, too small a sample, **peeking**
- Can't randomise? Quasi-experiments (difference-in-differences, regression discontinuity), with weaker assumptions
- **Uplift modelling**: target the **persuadables** (people the action actually changes), not the lost causes or the sure things

7. The decision: rule, ML or LLM

✎ DIAGRAM



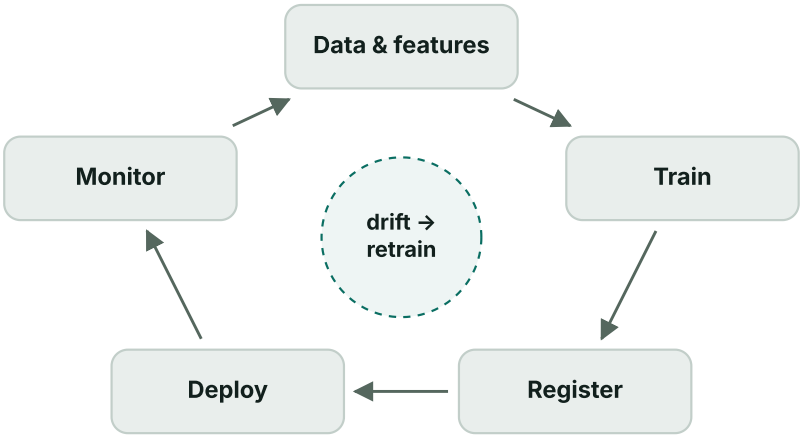
Ask in order, and always ask: would something cheaper be nearly as good?

8. MLOps for classical models

1. Features produced by a scheduled pipeline
2. Train → validate → **register** the model with its data version and metrics
3. Deploy as **batch** scoring (nightly scores to a table, e.g. churn) or **real-time** scoring (an API, e.g. fraud)
4. Monitor → retrain on a schedule or when drift crosses a threshold

- **Feature store**: features defined once and served the same way for training and live scoring, preventing **training-serving skew**
- **Model registry** and **experiment tracking** (MLflow is the common name)
- **Data drift** = inputs change · **Concept drift** = the relationship changes · **Label delay** = true outcomes arrive weeks later, so monitor inputs and prediction distributions meanwhile · **Feedback loops** = the model's own actions change future data

✎ DIAGRAM



The classical ML lifecycle: monitoring closes the loop back to retraining.

SAY IT

"If the problem is a spreadsheet, my default is a boosted tree: it costs a fraction of an LLM, it's usually more accurate on tabular data, and I can explain it to a regulator."

Data Foundations & Readiness

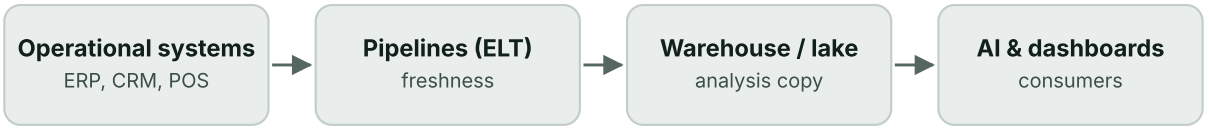
1. Why AI projects die

Post-mortems converge on the same cause: not algorithms but **data**. Data that doesn't exist, can't be accessed, can't be joined, can't be trusted, or can't legally be used. The discipline is sequencing: data readiness first, ambition second.

2. The data estate

- **Operational systems** (ERP, CRM, POS): where data is born, built for transactions, not analysis
- **Warehouse** (BigQuery, Snowflake): structured, governed copy for analysis · **Lake**: cheap raw storage, can become a swamp · **Lakehouse**: both
- **Pipelines** (ELT plus orchestration): where freshness is won or lost
- Ask: where does it live, how fresh is it, who owns it, can we query it without touching production?

✎ DIAGRAM



Data is born in operational systems and copied into an analytical layer; AI should read from the copy, not production.

3. Quality: six testable dimensions

- **Accuracy · completeness · consistency · timeliness · uniqueness · validity**
- Each is measurable, so each can be an automated test
- **Data contract**: an enforced agreement on schema, meaning, freshness and quality, so an upstream team can't silently break a model by renaming a column

4. The data-readiness assessment

A 1 to 3 week fixed-fee diagnostic, run before (or as phase zero of) any AI engagement. Score each candidate use case red / amber / green on:

1. **Existence & history**: captured at the right grain, far enough back?
2. **Quality**: profile it for a day; it reveals more than a week of meetings
3. **Accessibility**: a queryable copy, or locked in a vendor silo?

4 **Joinability**: one customer ID across systems, or three? (identity resolution can be a hidden multi-month project)

5 **Rights & permissions**: may it legally be used for this purpose?

✎ DIAGRAM

Use case	Exists	Quality	Access	Joins	Rights
Churn model	Green	Amber	Green	Red	Green
Support RAG	Green	Green	Amber	Green	Amber

Example readiness scorecard. Greens proceed, ambers get scoped fixes, reds get honest bad news early.

5. The permissions trap

> ⚠ A RAG assistant that indexes every document and answers anyone can leak documents to people who were never allowed to see them. Retrieval must respect each user's permissions; raising this early is a credibility moment.

6. Red flags in the room

- 'It's all in the warehouse' with no specifics · finance and marketing quoting different revenue numbers (no semantic layer) · key data in a vendor tool with no API · nobody owns the pipeline

7. Text-to-SQL: talking to your data

The most requested 'AI on our data' use case, and easy to get confidently wrong. The hard part isn't SQL, it's **meaning**.

1 User asks a question

2 **Retrieve the relevant schema** (a few tables and columns with descriptions), not the whole warehouse

3 LLM writes SQL, guided by example question → SQL pairs

4 **Validate**: parses, read-only, allowed tables only, LIMIT, dry-run cost check

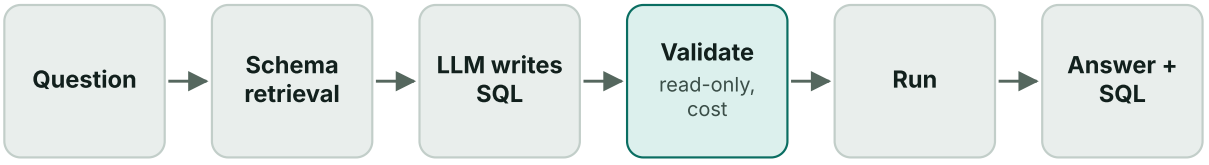
5 Run it

6 Answer in words and **show the SQL and the numbers**

- Fails on: ambiguous metrics ('revenue': gross or net?), wrong joins that duplicate rows, missing filters (test accounts), expensive scans

- Fixes: a **semantic layer** (one definition per metric), clarifying questions, a read-only account with row-level security
- Evaluate with **execution accuracy**: does the query return the right result? Never compare SQL text

✎ DIAGRAM



Validation before execution is what makes text-to-SQL safe to put in front of business users.

SAY IT

"Before scoping the AI, I'd run a short data-readiness check on each use case, and for text-to-SQL I'd put a semantic layer underneath, because the hard part is agreeing what revenue means."

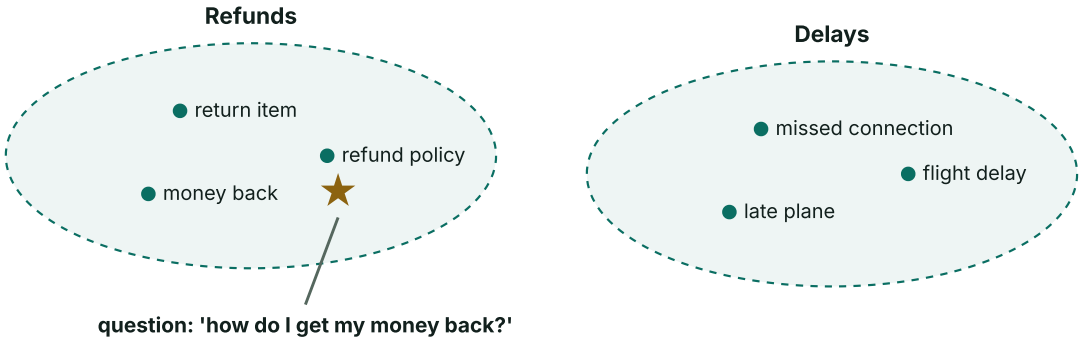
Embeddings, Retrieval & Vector Search

1. What an embedding is

An embedding is a fixed-length **list of numbers that captures meaning**. It's learned with **contrastive training**: the model sees pairs that belong together (a question and its answer) and pairs that don't, until belonging together means being close together. Once meaning is geometry, search becomes arithmetic.

- Similar meaning → vectors land **close together**
- Embeddings are **one-way**: you can't turn a vector back into text, which is why the original text is stored next to it

✎ DIAGRAM



Similar meanings sit close together, so the question lands in the refund cluster even with different words.

2. The retrieval flow

1. Chunk the documents
2. Embed each chunk
3. Store vector + **original text** + metadata in a vector database
4. Embed the user's question
5. Find the nearest vectors (cosine similarity)
6. Pass their **text** to the LLM

- **Cosine similarity** is the near-universal text default; always use the metric the model's documentation specifies (a mismatch silently degrades retrieval)
- For Arabic or mixed corpora use an explicitly **multilingual** model, which also gives cross-lingual search (an English question finds an Arabic document)

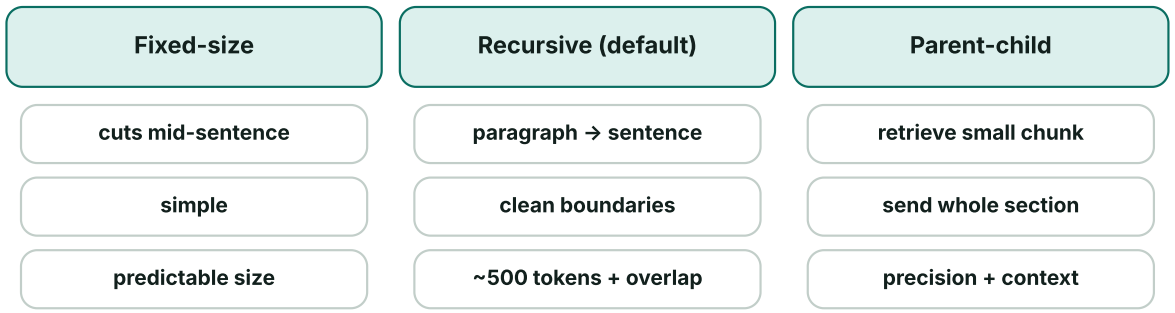
3. Chunking: the decision that sets the ceiling

The chunk is the unit you retrieve. Too big blurs several topics into one vector; too small loses context (a chunk saying '14 days' with no product name). **Most bad RAG is bad chunking.**

1. **Fixed-size**: every N tokens + overlap. Simple, cuts sentences in half
2. **Recursive** (default): split on paragraphs → sentences → words until under the limit
3. **Sentence / paragraph**: natural boundaries, uneven sizes
4. **Structure-aware**: split on headings, sections, slides, functions. Best for manuals, policies, code
5. **Semantic**: embed sentences, cut where meaning shifts. Costlier; test it
6. **Parent-child (small-to-big)**: retrieve small precise chunks, send the whole parent section to the LLM
7. **Sentence window**: retrieve one sentence plus its neighbours
8. **Contextual chunking**: prepend an LLM-written line saying where the chunk sits ('From the 2026 refund policy, electronics section')
9. **Proposition**: an LLM rewrites text into small standalone facts. Precise, expensive
10. **Late chunking**: embed the whole document first, then split, so each chunk keeps document context

- Knobs: size ~200 to 1,000 tokens (default 400 to 600), **10 to 20% overlap**, within the embedding model's input limit
- Metadata on every chunk: source, title, **heading path**, date, language, permissions
- Awkward content: keep **tables** whole or one row per chunk with headers repeated; fix **PDF parsing** first; **FAQs** as one question + answer per chunk; **code** by function

✎ DIAGRAM



Start recursive; switch to structure-aware or parent-child if the golden set shows chunks lacking context.

✎ DIAGRAM

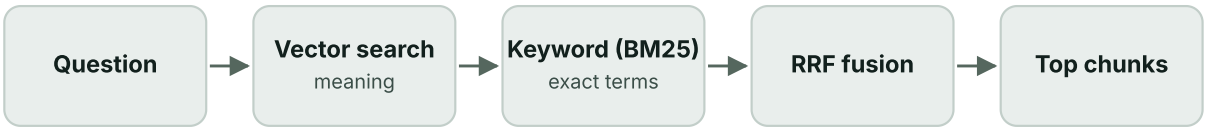
Symptom	Likely fix
Answer split across two chunks	More overlap, or parent-child
Chunk makes no sense alone	Heading breadcrumbs or contextual chunking
Tables come back garbled	Structure-aware parsing
Right document, wrong section	Smaller chunks plus reranking

Chunking problems show up as recognisable symptoms. Compare configs on recall@K.

4. Hybrid search

- Embeddings match meaning but **miss exact strings**: product codes, error codes, names, acronyms
- **BM25** (keyword search) catches exactly those
- Run both and merge with **reciprocal rank fusion (RRF)** → typically a 10 to 20% recall lift at trivial cost. Pure vector search is an anti-pattern at scale

✎ DIAGRAM



Hybrid search runs both searches and merges the ranked lists.

5. Vector databases and indexes

- Options: **pgvector** (default if Postgres is already there), **Qdrant** (open source, strong filtering), **Pinecone** (zero-ops). Pick by operational fit
- **Flat**: exact, checks everything (small sets) · **IVF**: searches the nearest clusters · **HNSW**: layered graph, fast, 95 to 99% recall, memory-hungry (the default) · **PQ**: compresses vectors
- **Metadata filters**: pre-filter (accurate) vs post-filter (can leave too few results) · **MMR** diversifies results so you don't get five near-duplicates
- **Bi-encoder** (separate embeddings, fast retrieval) vs **cross-encoder** (reads both together, accurate reranking) vs **ColBERT** (word-level, a middle ground)

6. Vector maths

- Dimensions: 384 · 768 · 1,024 · 1,536 · 3,072. More = more nuance, but more storage and slower search. **Matryoshka** embeddings can be shortened with little loss
- Storage = vectors × dimensions × **4 bytes** (float32)
- **Quantisation** shrinks it: float16 = ½, int8 = ¼, binary = 1/32, with some recall loss

✎ DIAGRAM

```
1,000,000 vectors × 1,536 dims × 4 bytes
= 6,144,000,000 bytes
≈ 6 GB (+ index overhead)
```

The storage sum to be able to do on a whiteboard.

7. Retrieval metrics

- **recall@K**: was the right chunk anywhere in the top K? (check this first)
- **precision@K**: how much of the top K was relevant?
- **MRR**: how high was the first correct result? · **nDCG**: were the best results ranked highest?

SAY IT

"An embedding is a list of numbers that captures meaning, so similar texts land close together. I'd chunk recursively at about 500 tokens with heading metadata, run hybrid search, and judge it on recall@K."

RAG Architecture

1. RAG in one sentence

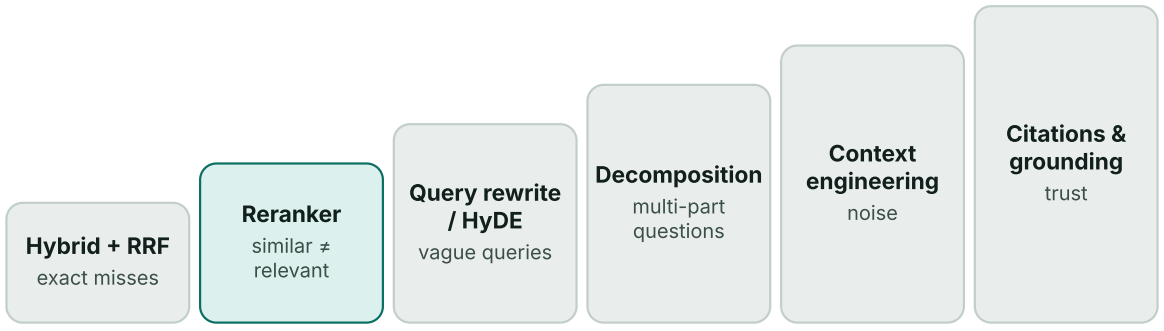
Retrieval-augmented generation: retrieve relevant chunks from your documents and give them to the model to answer from. It changes what the model *knows at answer time*, so updating knowledge means updating the index, not the model.

2. Naive RAG breaks four ways

1. **Vague queries** retrieve bad chunks (conversational questions sit far from document language)
2. **Similar ≠ relevant**: the most similar chunks often don't contain the answer (unreranked top-K carries 20 to 40% noise)
3. **Exact-match misses**: SKUs, table names, error codes
4. **Noisy or contradictory context**: five chunks in, two relevant, and the model confidently synthesises from the noise

3. The production ladder (build in this order)

✎ DIAGRAM



Each rung fixes a named failure. Measure after each one; the reranker is usually the biggest single gain.

Present RAG as targeted fixes for named failures, not as fashion.

- **Reranker (cross-encoder)**: reads the question and each candidate together to score true relevance. Retrieve ~50 cheaply, rerank to the best 5. Commonly +10 to 20% quality
- **HyDE**: a cheap model drafts a plausible answer and you search with *that* embedding, because answers sit near answers
- **Context engineering**: best chunk first, filter hard (three good chunks beat five mediocre), dedupe, number the chunks so the model can cite them

- **Grounding & citations**: answer only from the chunks, cite each claim, say 'I don't know' otherwise

4. Why citations are worth paying for

- **Trust**: users can verify · **Debuggability**: trace a wrong answer to the chunk that caused it · **Quality**: enables automated faithfulness checks · **Compliance**: an audit trail

5. Evaluate in three places

- **Retrieval** → recall@K on a hand-labelled question set (the most important number)
- **Context** → right chunks, right order
- **Generation** → faithfulness (every claim traceable to a chunk), scored by a calibrated LLM judge

6. Debugging a confidently wrong answer

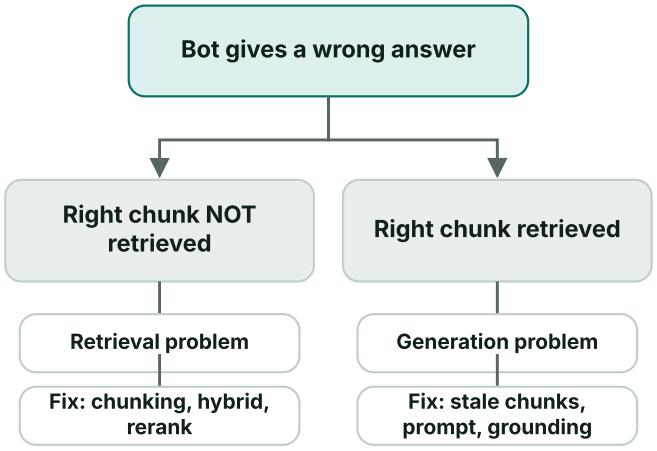
1 **Log what was retrieved** for that exact question

2 Right chunk missing → **retrieval** problem: parsing, chunking, embedding model, top-K, hybrid search, reranker

3 Right chunk present, answer still wrong → **generation** problem: conflicting or stale chunks, weak grounding instructions, model ignoring context

4 Turn the failure into a permanent test case in the golden set

✎ DIAGRAM



Split retrieval failures from generation failures before touching anything.

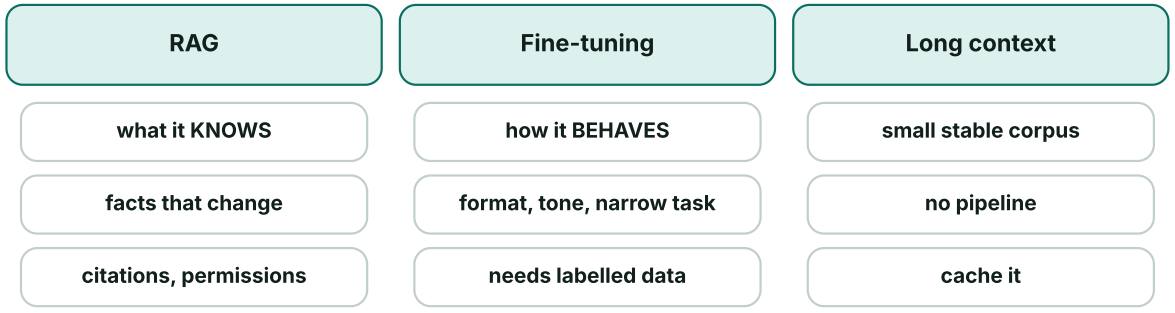
7. RAG vs fine-tuning vs long context

- **RAG = knowledge** (facts that change, need citations, per-user permissions)

- **Fine-tuning = behaviour** (format, tone, a narrow task, or distilling a big model into a small cheap one)
- **Long context + caching** = a small (under ~500k tokens), stable corpus with no pipeline to build
- **Prompting first** for both. They combine: a fine-tuned small model on top of RAG is common
- Long-context tax: a 200k-token call costs and takes roughly 10× a 20k call, even when it fits
- True cost of fine-tuning: the labelled data, the eval set to prove it helped, retraining on every base-model change, and lock-in

> ⚠ Fine-tuning to 'teach the model our documents' fails: facts get blurred, can't be cited, go stale, and can't be removed per user.

✎ DIAGRAM



RAG for knowledge, fine-tuning for behaviour, prompting first for both.

8. How models are trained and tuned

1 **Pre-training**: predict the next token over trillions of tokens → knowledge, not manners

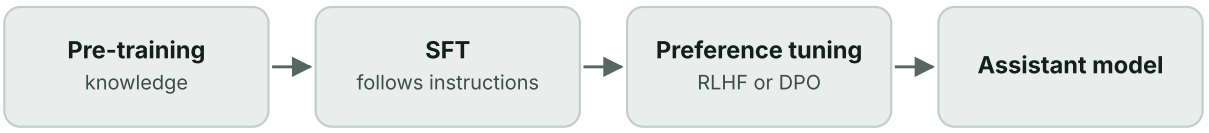
2 **SFT** (supervised fine-tuning): example prompts + ideal answers → follows instructions

3 **Preference tuning**: **RLHF** (train a reward model on human rankings, then optimise against it) or **DPO** (learn directly from better-vs-worse pairs, simpler)

- **LoRA**: train small add-on matrices instead of all weights · **QLoRA**: LoRA on a quantised base, fits big models on modest GPUs · **Distillation**: a big model teaches a small one a narrow task
- **Quantisation**: weights in fewer bits (16 → 8 → 4), less memory, faster, small quality loss

- Risk: **catastrophic forgetting** (narrow tuning degrades general skills)

✎ DIAGRAM



How a raw next-token predictor becomes a helpful assistant.

SAY IT

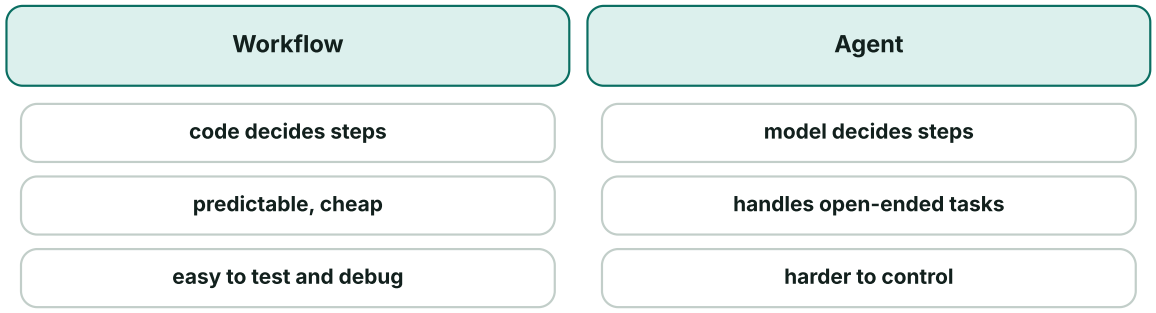
"RAG for knowledge, fine-tuning for behaviour, prompting first for both. When the bot is wrong, I first check what was retrieved, because that tells me whether it's a retrieval or a generation problem."

Agents & Orchestration

1. Definitions

- **Agent**: a model in a loop with tools. It gets a goal, picks an action, observes the result, and repeats until done or a stop condition fires. **The model decides the path**
- **Workflow**: a predefined sequence where **code decides the path** and LLM calls fill specific slots (classify, extract, draft)
- **Workflows are the default.** Agents earn their place only when the path genuinely can't be known in advance (open-ended research, operating software whose state must be discovered)

✎ DIAGRAM



'We built an agent' isn't a boast when a five-step workflow would do the job at a tenth of the cost.

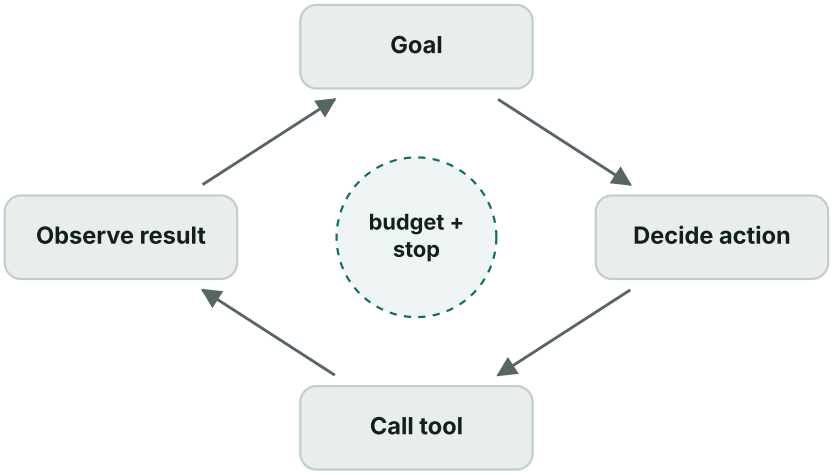
2. Tools

- The model chooses tools by reading their **names and descriptions**, so a vague description is a routing bug. Agent quality is mostly tool quality
- Few tools, sharply described, typed narrow inputs, errors written so the model can act on them
- **Idempotent** tools: safe to call twice, because retries happen
- **MCP** (Model Context Protocol) standardised how tools are plugged in, but every connected server widens the security surface

3. Loop hygiene

- Explicit **budgets**: max steps, tokens, spend
- **Stop conditions** and **checkpoints** that save state
- **Human approval** before anything consequential or irreversible: sending messages, moving money, deleting data, committing code
- Ask of every tool: what's the worst thing this can do, and who approves it?

✎ DIAGRAM

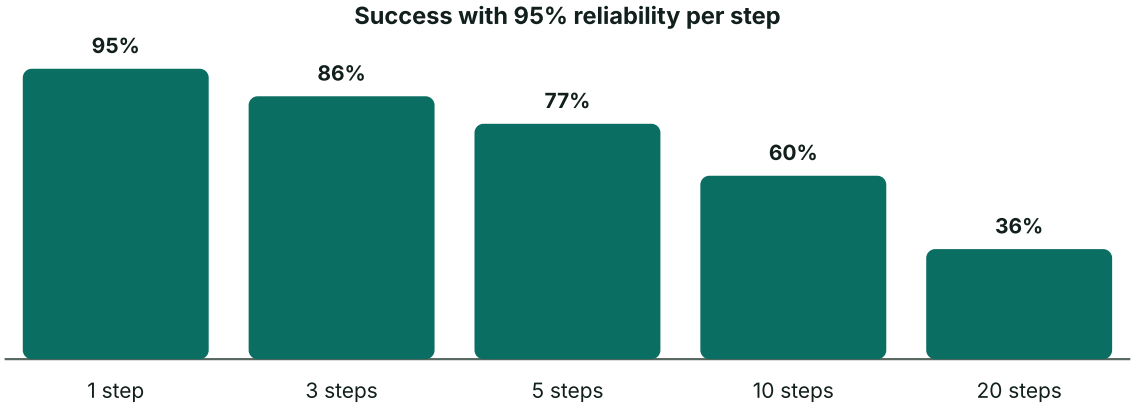


The agent loop. Put a human gate on any arrow that moves money, sends messages or deletes data.

4. Why agents fail on long tasks

- **Compounding error**: 95% reliability per step is only ~60% over ten steps
- Fix with structure, not hope: shorter verified segments, external checkpoints, clean context, capped steps
- Measure **task success rate** on realistic end-to-end tasks, plus cost and steps, and review failed trajectories

✎ DIAGRAM



Errors multiply across steps, which is why long agent runs need checkpoints and verification.

5. Context windows and agent memory

- The **context window** is everything the model sees in one call: system prompt, tools, history, retrieved documents and its own output. Treat it as a **budget**, not a bucket
- Long chats: **sliding window** (last N turns), **summarisation** of older turns, **compaction** (rewrite history shorter); keep stable content at the top for prompt

caching

- Memory types: **short-term** (this task), **long-term** (facts and past episodes retrieved when relevant), **profile** (stable user facts), **procedural** (learned how-to)
- Design questions: who decides what's saved, how wrong memories are corrected, what must never be stored, how one bad memory is stopped from spreading

✎ DIAGRAM



Every block competes for the same window. Summarise history and retrieve only what's relevant.

The context window as a budget.

SAY IT

"I default to a workflow and only use an agent when the steps genuinely can't be known in advance, with step budgets, checkpoints and human approval on anything irreversible."

Evaluation & Quality Systems

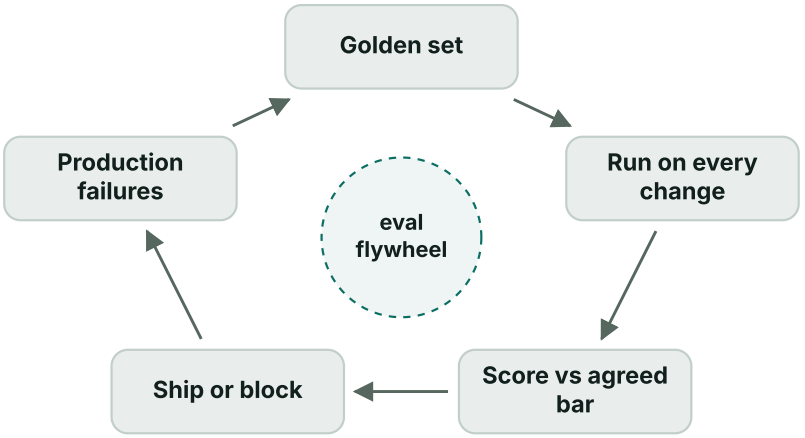
1. Why evals are the centre of gravity

Evaluation is the **control system** of AI engineering. Without it you can't choose a model, approve a prompt change, or answer the only question executives ask: 'is it good, and is it getting better?'. Amateurs build and eyeball; professionals define what good means, encode it as a suite, and let the suite gate every change. In a statement of work, an eval threshold turns 'is it done?' from an argument into a measurement.

2. The golden set

- **30 to 100 real, hand-labelled examples** to start: real inputs with expected outcomes
- Deliberately mix easy cases, hard cases and known failure modes
- **The flywheel**: every bad production output becomes a permanent test case, so the same failure can never return unnoticed

✎ DIAGRAM



Failures from production feed back into the golden set, so the suite only grows.

3. Scoring

- **Exact checks** wherever possible: schemas validate, SQL returns the right result, code passes tests. Free and unarguable
- **LLM-as-judge** for open-ended qualities (helpfulness, faithfulness, tone): a strong model grades against a written rubric
- Judge biases: **position** (favours the first answer: swap and average), **verbosity** (rewards length: tell it not to), **self-preference** (favours its own family: use another provider), **rubric drift** (vague rubrics score vibes)

- **Calibrate** the judge against human labels and only trust it once agreement is high

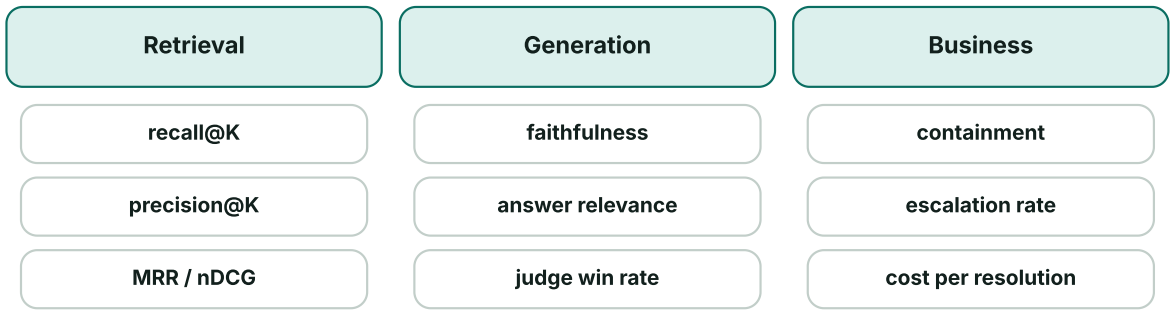
4. Gating changes

- Run the suite in **CI on every change**: prompt, model, chunking, documents, settings
- Agree the **pass bar** up front, so shipping is a rule, not a judgment call
- Below the bar → the change is blocked, not rolled back after it's live
- After launch: **online evals** on a sample of live traffic, plus thumbs-down, escalations and drift monitoring

5. Metrics by layer

- **Retrieval**: recall@K, precision@K, MRR, nDCG
- **RAG (RAGAS family)**: **context precision** (was the context relevant?), **context recall** (did it contain everything needed?), **faithfulness** (claims backed by context?), **answer relevance** (does it answer the question?)
- **Generation**: exact match / F1 for short facts; **BLEU / ROUGE** are word-overlap scores and weak for open-ended answers; **pointwise** (score one answer) vs **pairwise** (A vs B, giving a win rate)
- **Classification**: precision, recall, F1, confusion matrix
- **Business**: **containment** (resolved without a human), escalation rate, resolution rate, CSAT, **cost per resolved case**

✎ DIAGRAM



Name a metric for each layer; 'we'd run evals' alone isn't an answer.

6. Rigour

- Enough examples to trust a difference (2 points on 30 questions may be noise)
- Outputs vary, so run several times and average
- Keep a held-out test set you never tune against

SAY IT

"I'd build a golden set from real questions, measure retrieval and answers separately, gate every change in CI against an agreed bar, and monitor live traffic after launch."

Security, Privacy & Adversarial Robustness

1. Prompt injection

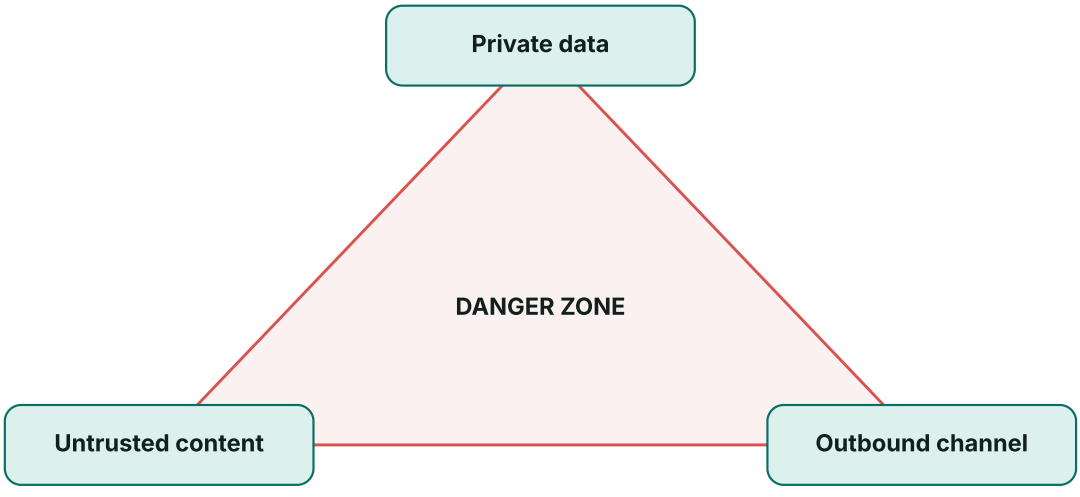
The most important security fact about LLM systems: the model **cannot reliably tell instructions from data**. Anything in the context (a user message, a retrieved document, an email, a web page, a tool result) may be treated as an instruction.

- **Direct injection**: the user tells the model to ignore its rules
- **Indirect injection** (the dangerous one): instructions hidden in content the system reads for the user. A CV telling the screening bot to rank it first; an email telling the assistant to forward the inbox
- As of today **no complete defence exists**. Saying so plainly is a credibility marker; anyone selling a guaranteed fix is selling snake oil

2. The lethal trifecta

Real danger concentrates where a system combines all three: **private data** + **untrusted content** + **a way to act or send outward**. Remove or human-gate any one leg and the catastrophic scenarios collapse. That's why the defences are architectural, not prompt-level.

✎ DIAGRAM



Cut any one side (remove data access, quarantine untrusted reading, or approval-gate the outbound action) and the worst cases go away.

The lethal trifecta.

3. The defence stack, strongest first

1. **Least privilege**: tools get only what the task needs
2. **Human approval** on consequential actions

3 **Privilege separation**: a trusted agent that never reads untrusted content directs a quarantined one that does

4 Treat all tool and retrieval output as **untrusted**

5 **Output / action filtering**: allow-list what agents may fetch and where they may send

6 **Injection classifiers**: useful but porous

7 **Logging and red-teaming**, with attack cases in the eval suite

4. Quieter risks

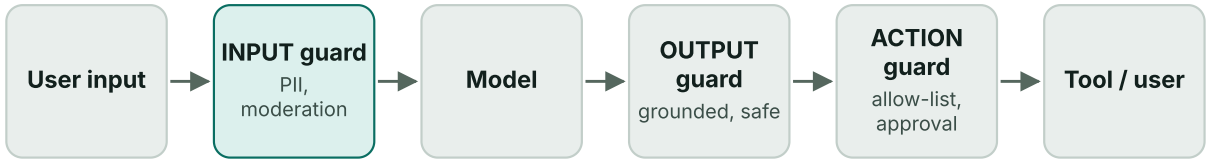
- **Data leakage** from over-permissioned retrieval (a security control, not a nicety)
- Sensitive data sent to third-party APIs without contractual cover
- **Jailbreaks**: mostly brand and abuse risk
- **Supply chain**: every MCP server and plugin is third-party code with data access. Vet it, pin it, limit it

5. Guardrails at input, output and action

- **Input** (before the model sees it): PII detection and redaction, moderation, injection detection, topic limits
- **Output** (before the user sees it): schema validation, **groundedness** (claims backed by sources), PII-leak and toxicity checks, business rules ('never quote prices')
- **Action** (before a tool acts): allow-lists, spend and volume limits, human approval
- Every check adds latency and cost, so **tier them**: cheap rules first, model checks only when needed

> ⚠ Over-strict guardrails make a useless product. Pair every block with a way forward: offer a callback, escalate to a human, point to the right page.

✎ DIAGRAM



Three checkpoints, each with a graceful fallback rather than a dead end.

SAY IT

> *"The defence is architectural: least privilege, human approval for risky actions, and never combining private data, untrusted input and an outbound channel. Guardrails sit at input, output and action, each with a way forward."*

Deployment & Production Engineering

1. The POC-to-production gap

Most AI pilots never reach production, and the cause is rarely the model. It's **reliability, integration, cost control, monitoring, security review and ownership**. A demo is the happy path; production is the happy path plus outages, rate limits, malformed inputs, adversarial users, cost spikes, deprecations, and the question of who gets paged. The deliverable that closes it is a **production-readiness review**: a red/amber/green checklist plus a remediation backlog.

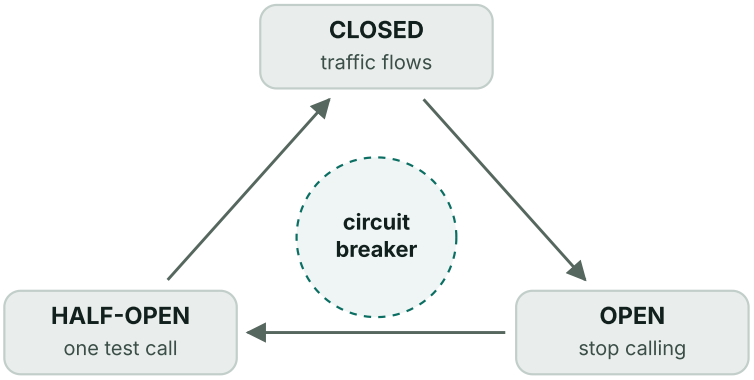
2. Serving: API vs self-host

- **Continuous batching** (adding and removing requests token by token to keep GPUs busy) is why API prices collapsed; **speculative decoding** (a small draft model proposes, the big one verifies) gives 1.5 to 3× faster decode
- Self-hosting cost = GPUs + engineering + **utilisation**. A single tenant's spiky traffic leaves GPUs mostly idle, so **APIs win on cost** unless volume is high and steady
- Real reasons to self-host: **data residency**, deep customisation, latency locality
- Middle path: hyperscaler **private endpoints** (Bedrock, Vertex AI, Azure AI Foundry) with in-region processing and a no-training data agreement

3. Reliability primitives

- **Timeouts** on every call · **Retries with exponential backoff + jitter** on transient errors (429, 5xx) · **Idempotency keys** so a retry doesn't double an action
- **Fallback chain**: primary model → secondary provider → honest degraded answer
- **Circuit breaker**: after N failures stop calling the failing provider (open), test with one request after a cool-down (half-open), resume on success (closed)
- **Bulkheads** (separate connection pools per dependency), **backpressure** (shed load when the queue is full), a **token-bucket** rate limiter, **graceful degradation** (serve cached or simpler answers)

✎ DIAGRAM



N failures → open; after a cool-down → half-open; a successful test → closed.

4. Shipping AI like software

1. Prompts, examples, tool schemas and **model-version pins** in git
2. **CI gate**: the eval suite runs on every change; regressions block the merge
3. Build a container, tag it with the commit
4. Staged rollout: staging → canary
5. **Instant rollback** to the pinned previous version

- Secrets in a secrets manager, never in code; autoscale on queue depth and concurrent requests, not just CPU

5. Release strategies

✎ DIAGRAM

Strategy	How	Use for
Canary	1 → 5 → 25 → 100% of traffic	Default for prompt and model changes
Blue-green	Full new copy, flip all at once	Clean cutovers with instant revert
Shadow	Copy traffic, don't show output	Testing a model swap on real traffic
A/B	Randomised cohorts, compare outcomes	Deciding which version is better

Canary bounds the blast radius; shadow lets you test on real traffic with zero user risk.

6. Observability and cost

- **Traces** (one ID stitching retrieve → rerank → generate → tool), **metrics** (p50/p95/p99 latency, error rate, tokens, cost per request, cache-hit rate), **logs** (prompt, completion, model version, outcome)
- Watch **p95**, not the average
- **LLM FinOps**: spend by feature and customer, budgets and alerts, max-token caps, a cache-hit-rate dashboard
- **Online evals** on a sample of live traffic; watch for **drift**

7. Streaming, APIs and protocols

- **Streaming** sends tokens as they're generated: it doesn't make generation faster, it makes the first words appear in about a second
- **SSE** (server-sent events): one-way, server → browser, plain HTTP, auto-reconnect. The standard for streaming LLM answers
- **WebSockets**: two-way persistent connection. For real-time voice and live collaboration
- Long jobs: **polling**, **webhooks**, or the **batch API** (~50% cheaper)
- Rate limits: **RPM** (requests/min) and **TPM** (tokens/min); a **429** means back off with jitter and queue

✎ DIAGRAM



One-way is SSE, two-way is WebSockets.

8. Incidents and on-call

- AI incident types: provider outage, silent quality regression, cost spike, safety incident, model deprecation deadline
- **SLOs** (p95 latency, availability, minimum quality score) with alerts and a **runbook** per alert

- **Blameless post-mortem**, and the fix that matters: add the failure to the eval set
- Model deprecation playbook: eval candidates → shadow-test → canary → re-tune → update pins

SAY IT

"A demo is the happy path. Production adds timeouts, retries with backoff, a fallback provider, a circuit breaker, eval-gated releases with canaries, and per-request cost and quality tracking."

Economics, TCO, Vendors & Procurement

1. Why AI business cases die in finance review

- Benefits stated as single points instead of ranges
- Baselines never measured
- Costs missing their biggest lines (especially change management)
- Benefits no owner will sign for

Finance's real question isn't 'is the number big?' but 'can I trust the person who built this?'.

2. The business case that survives

1 **Baseline first**: measure today's process (minutes per ticket, cost per document) before promising to improve it

2 **Benefits as ranges with owners**: conservative / expected / optimistic, each assumption named, signed by the P&L owner who'll bank it

3 **Full costs**: build, run (tokens × volume × language multiplier), integration, **change & enablement**, evals and governance, ongoing operations

4 **Sensitivity**: show which assumption moves the answer most (usually adoption or deflection rate, almost never token price) and its break-even value

✎ DIAGRAM



Show a range with a break-even line: 'it works as long as adoption is above X'.

3. 'AI is too expensive'

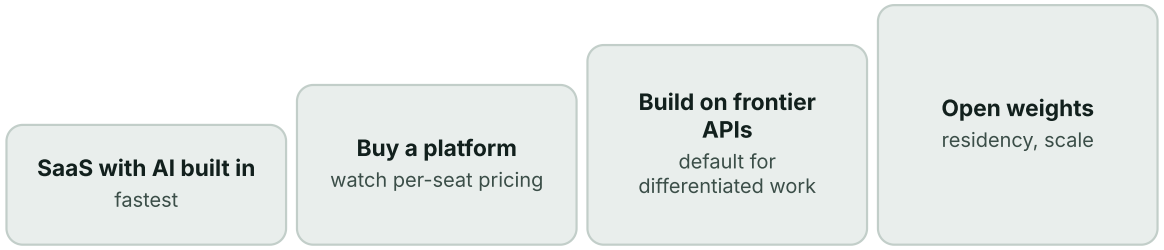
- Reframe from sticker price to **unit economics**: cost per resolved ticket or per analyst-hour saved, compared with today
- Most 'too expensive' AI is **unoptimised** AI: no caching, no routing, verbose outputs

4. Tech case maths

- **TCO** = purchase or build + implementation + training + ongoing run and support
- **ROI** = (benefit − cost) ÷ cost
- **Payback** = upfront cost ÷ annual savings (e.g. $6M upfront, $4M/year saved → 1.5 years = 18 months)

5. The build-buy ladder

✎ DIAGRAM



Moving up: more control and effort, less lock-in to a vendor's roadmap.

Buy what's commodity, build what's differentiating.

- Tiebreaker: is the capability **differentiating**? Buy commodity (meeting notes, generic copilots); build what encodes your process, data or customer advantage
- Stress-test any usage-priced vendor at **5× projected volume**

6. What to negotiate

- **Data terms**: a DPA, no-training commitment, retention, residency
- **Model lifecycle**: deprecation notice and migration support
- **SLAs**: uptime, rate limits, incident response · **Exit**: data export, transition help
- Score vendors on **your own evals**, never their demo, with a written scorecard (capability, cost at volume, data terms, roadmap, lock-in)

SAY IT

"I'd build the case from a measured baseline, show benefits as a range signed by an owner, include change management as a cost line, and show the break-even on adoption."

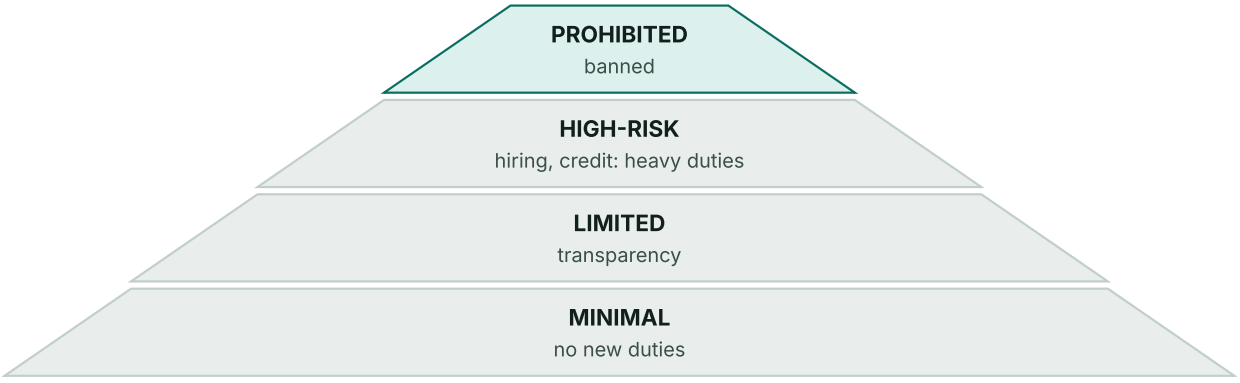
Governance, Risk & Responsible AI

1. The EU AI Act: risk tiers

The anchor regulation every other regime borrows from. It's **extraterritorial**: it applies to any system whose outputs are used in the EU.

- **Prohibited**: social scoring, manipulative techniques, most real-time biometric surveillance (banned since early 2025)
- **High-risk**: hiring, credit, insurance, education, critical infrastructure. Allowed with heavy obligations (risk management, data governance, documentation, human oversight, conformity assessment). Originally due August 2026; the 2026 AI Omnibus moved stand-alone systems to **2 December 2027** and AI in regulated products to **2 August 2028**
- **Limited-risk**: transparency (people must know they're talking to AI; synthetic content disclosed)
- **Minimal-risk**: most applications, no new obligations

✎ DIAGRAM



Obligations scale with risk. Classify the use case first, because the tier decides the obligations.

2. Frameworks clients ask about

- **NIST AI RMF**: voluntary US framework, four functions: **Govern, Map, Measure, Manage**
- **ISO/IEC 42001**: the certifiable AI management system (the ISO 27001 of AI)
- Gulf: **UAE PDPL** (plus DIFC and ADGM rules) and **Saudi PDPL** with data-transfer restrictions, so data residency is a real architectural constraint

3. Minimum viable governance (weeks, not years)

1. An **AI inventory**: every system, purpose, risk class, owner and data touched, including **shadow AI**
2. Risk classification at intake
3. Documentation by design (model cards, data provenance, eval results)
4. Human oversight proportionate to risk
5. Fairness testing where systems touch people's opportunities
6. Incident response
7. A review forum with authority to say no

Framed right, governance is an enabler: the organisations that deploy fastest are the ones whose governance lets legal and security say yes quickly.

4. Fairness and bias

- Where bias comes from: **historical** (past unfair decisions), **representation** (groups under-represented), **measurement** (a poor proxy label), and **proxy variables** (postcode standing in for a protected trait even after it's removed)
- **Demographic parity**: similar positive-outcome rates across groups
- **Equal opportunity**: qualified people approved at similar rates (equal true-positive rates)
- **Equalised odds**: similar true-positive and false-positive rates

> ⚠ These definitions conflict mathematically; choosing one is a policy decision for the client, not a technical default.

- Measure outcomes and error rates **per group**; mitigate before, during or after training; keep human review and an appeal route
- **Explainability**: global (what drives the model overall) vs local (why this person: **SHAP values**, **counterfactuals** like 'approved if income were X higher')
- For LLMs, test with paired prompts that differ only in a name or dialect

✎ DIAGRAM



Fairness is measured per group, chosen as policy, and backed by explanations people can challenge.

SAY IT

"First I'd classify the use case by risk tier, because that decides the obligations. For anything touching people's opportunities I'd measure outcomes per group, agree which fairness definition applies, and make each decision explainable."

Consulting Craft & the Case Method

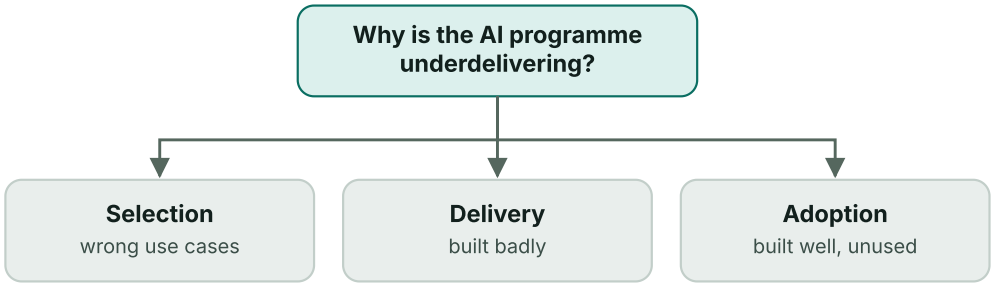
1. Hypothesis-first problem solving

The amateur gathers everything and hopes insight emerges. The consultant states a **testable best guess immediately** ('AI spend tripled: my hypothesis is untiered routing plus no caching'), structures the analysis to confirm or kill it fast, and **updates out loud** when data disagrees. The loop (stated, tested, revised) is the performance, whether or not the first guess was right.

2. Issue trees and MECE

- Decompose the problem into branches that are **mutually exclusive** (no overlap) and **collectively exhaustive** (nothing missing)
- Example: 'Why is the AI programme underdelivering?' → **selection** (wrong use cases) / **delivery** (built badly) / **adoption** (built well, unused)
- Pressure test: can you say what each branch excludes?

✎ DIAGRAM

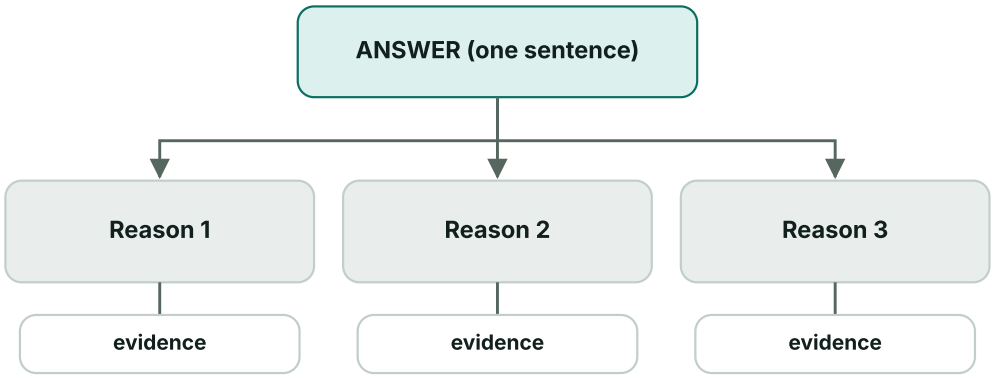


A MECE issue tree: no overlap, nothing missing.

3. The Pyramid Principle

- **Answer first**, then grouped reasons, then evidence
- 'We recommend pausing the rollout: adoption is stalled at 9%, the root cause is workflow mismatch, and a four-week redesign recovers the business case.' Then the detail
- In an interview: one-sentence answer, then 'for three reasons', then give them

✎ DIAGRAM



Executives read the first line and decide whether to read the second.

4. Discovery: value × feasibility × risk

- **Value**: in the client's own arithmetic (volume × time × loaded cost), with a named owner
- **Feasibility**: data readiness, integration surface, pattern maturity, change surface
- **Risk**: cost of an error, regulatory class, reversibility
- The portfolio: 2 to 3 **quick wins** (fund credibility), 1 to 2 **strategic builds**, a **watch list**, and a **kill list** with reasons

✎ DIAGRAM



Start top-right; the first project's job is to earn trust.

5. The three-altitude drill

Explain the same reality to three audiences without condescending:

- **CFO**: money and control ('you're billed by the word, and words written back cost ~5× words read, so tightening verbosity halves the bill')
- **Data lead**: drivers and levers ('output tokens dominate; cut output, cache the prefix, route easy queries')

- **Engineer**: mechanism and configuration ('decode reads the full KV cache per token; set max_tokens, move stable content above the cache breakpoint')

SAY IT

"My hypothesis is X. To test it I'd look at A and B; if A comes back wrong, I'd switch to Y."

General Case Craft

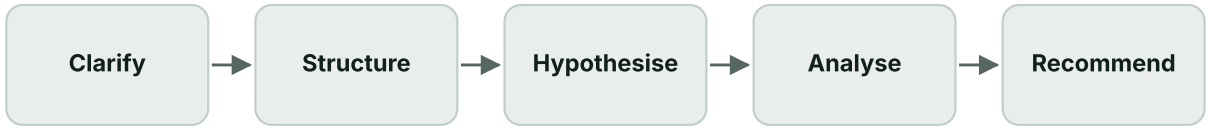
1. What a case interview really tests

A simulation of the job, scored on **structure, communication and composure** far more than the 'right' number. A candidate who structures beautifully and hedges the deep tech beats one who knows everything and rambles.

2. The five-step flow

1. **Clarify**: play the question back, ask 2 to 3 sharp questions (objective, success metric, timeframe, what's been tried)
2. **Structure out loud**: a MECE tree built for *this* problem
3. **Hypothesise**: a falsifiable best guess and the analysis that would test it
4. **Analyse with visible maths**: state assumptions, round hard, narrate
5. **Recommend answer-first**: recommendation, reasons, risks, next step

✎ DIAGRAM



Use the flow as a container you think inside.

3. Estimation

- **Top-down**: start from a big known number (population, households) and narrow with ratios
- **Bottom-up**: build from units (sales per store × number of stores)
- State every assumption, segment where ratios differ, round aggressively, finish with a **sanity check** per person or per unit
- Anchors: world ~8B · large country 30 to 70M · ~2.5 people per household · ~4 characters per token

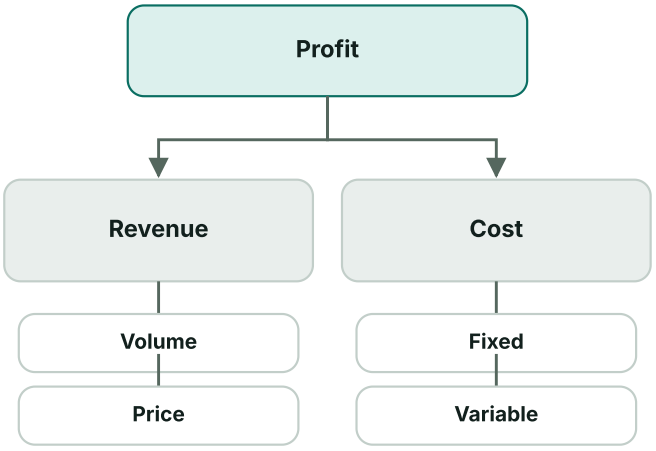
4. Core frameworks (adapt, never recite)

- **Profitability**: Profit = Revenue − Cost; Revenue = volume × price; Cost = fixed + variable. If revenue is flat, look at cost and mix
- **Market entry**: is it attractive? can we win? do the economics work? how do we enter (build, buy, partner)? End with go / no-go and the conditions

- Lenses: **3Cs**, **4Ps**, **Porter's Five Forces**, **value chain**

> ⚠ A structure built for the problem beats a memorised framework forced onto it.

✎ DIAGRAM



The profitability tree unlocks a huge share of business cases.

5. Mental maths that survives pressure

- **Strip the zeros**, multiply the small numbers, put the zeros back
- **Split awkward numbers**: × 18 = × 10 + × 8; × 250 = × 1,000 ÷ 4; × 1.5 = the number plus half of it
- Percentages from **10%**: 15% = 10% + half of it; 35% = 3 × 10% + half of 10%
- Count zeros out loud, and check you finished the question (annual? multiply by 12)
- Months: half a year = 6, a third = 4, a quarter = 3

✎ DIAGRAM

```
400,000 × 18
→ 4 × 18 = 4×10 + 4×8 = 40 + 32 = 72
→ put back 5 zeros
= 7,200,000
```

The zero trick, worked.

6. Behavioural and fit (STAR)

- **Situation · Task · Action** (most of the time, say 'I') · **Result** (with a number)
- Prepare 5 to 6 stories, each mapped to 2 to 3 likely questions, rehearsed to ~90 seconds

- Pick stories that position you: a right-sizing decision, a measured quality climb, an adoption save, bad news delivered early with options

SAY IT

"Let me take 30 seconds to structure this." Then give 3 or 4 branches and say why each matters.

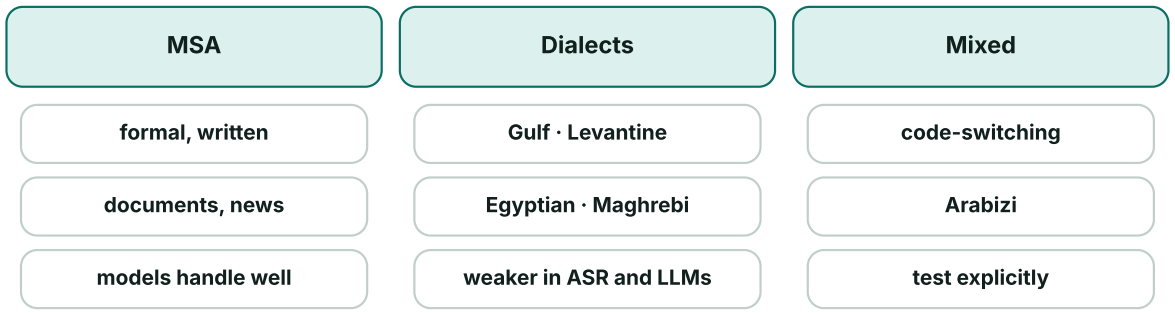
Multimodal, Voice & Arabic AI

1. The Arabic language landscape

In the Gulf, 'does it work in Arabic?' is often the deciding question.

- **MSA** (Modern Standard Arabic): formal, written, news and documents
- **Dialects**: Gulf, Levantine, Egyptian, Maghrebi: how people actually speak and type
- **Code-switching**: Arabic and English mixed in one sentence
- **Arabizi**: Arabic in Latin letters and numbers ('7abibi')

✎ DIAGRAM



Test the variety your users actually write and speak.

2. Technical issues with Arabic

- **Token tax**: ~1.5 to 2.5× English tokens for the same meaning → more cost and latency, less context
- **Normalisation** before keyword search: alef forms (ا أ إ آ), ta marbuta, diacritics, elongation
- Right-to-left and mixed-direction display
- **Speech recognition** is noticeably weaker on dialects
- Retrieval: a **multilingual embedding model**, cross-lingual testing, normalised text for BM25

3. Evaluating Arabic

- Golden sets **per dialect**, labelled by native speakers
- LLM judges can favour MSA or English phrasing, so calibrate them on Arabic
- Report quality by language and dialect, not one overall score
- Arabic-centric models exist (e.g. **Jais**, **ALLaM**): test them alongside global models on your data
- Residency: UAE and Saudi **PDPL** often require in-region processing

4. Documents and vision

- **OCR**: turns images of text into text. Cheap and fast, loses layout, struggles with tables and handwriting
- **Document AI** (Textract, Azure Document Intelligence, Google Document AI): keeps structure, tables and key-value fields
- **Vision-language models (VLMs)**: multimodal LLMs that read images directly. Flexible on messy layouts, cost more per page
- Fixed high-volume forms → document AI; varied messy documents → VLM into a schema; search → parse with structure, then chunk
- Extract into a **strict schema**, return **confidence per field**, route low-confidence fields to a human, link every value to its page, measure **field-level accuracy**

✎ DIAGRAM



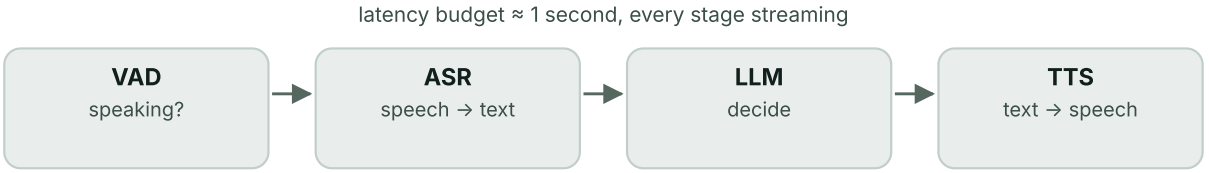
Combine cheap extraction with a VLM for the hard pages, and keep a human on uncertain fields.

5. Voice AI: the real-time pipeline

1. **VAD** (voice activity detection): is someone speaking?
2. **ASR** (speech-to-text)
3. **LLM** (with tools and RAG)
4. **TTS** (text-to-speech)

- **Speech-to-speech** models go audio-in, audio-out: lower latency and keep tone, but harder to control and evaluate
- **Latency budget ~1 second**: stream every stage (partial transcripts, first LLM tokens straight into TTS), short spoken-style answers
- Feels human: **barge-in** (the user can interrupt), good turn detection, noise and dialect handling, reading critical details back
- Transport: **WebSockets or WebRTC** (two-way audio), not SSE
- Metrics: **word error rate per dialect**, p95 response latency, task completion, transfer-to-human rate

✎ DIAGRAM



A voice agent is a latency budget, plus barge-in so callers can interrupt.

SAY IT

"For Arabic I'd budget a token multiplier, use a multilingual embedding model with normalisation, and evaluate per dialect. A voice agent is a latency budget: VAD, streaming ASR, LLM and streaming TTS over WebSockets, with barge-in."

Interview playbook (Frameworks)

1. Spot the format from the first sentence

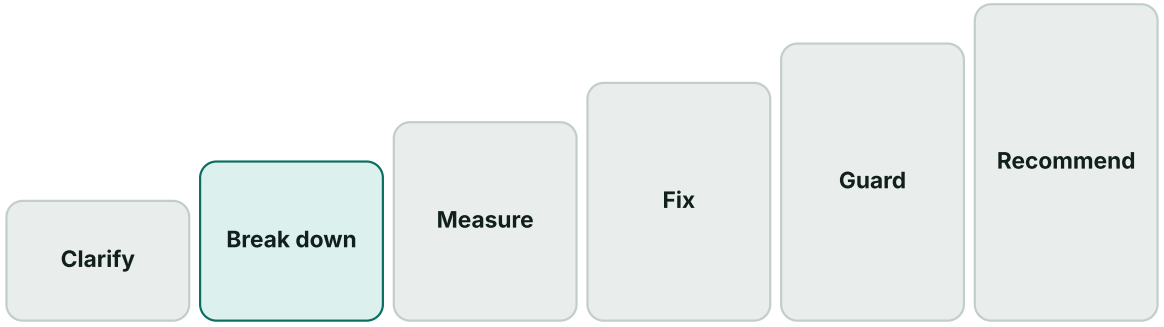
- 'It's broken / slow / expensive / unsafe' → **Fix it**
- 'Design / build / architect' → **Design it**
- 'Should we / what's your view' → **Business case**
- 'What is / compare' → **Explain it**
- 'How many / how much' → **Estimate**
- 'Tell me about a time' → **Behavioural**
- Data pack + deadline → **Written case** · recruiter call → **Screening**

> ⚠ Watch the ending: a number means estimate; 'should we' or 'what's your view' means business case; a symptom in an existing system means fix it.

2. Fix it: C · B · M · F · G · R

1. **Clarify**: restate, ask what's built, what's in place, what success means
2. **Break it down**: never skip. 'This is a cost problem, so I'll break it into...'
3. **Measure**: ask for data per piece, find the piece that moved ('the drop was sudden, so what changed?')
4. **Fix**: cheapest and safest first
5. **Guard**: every change passes the golden-set evals before shipping; monitor after
6. **Recommend**: answer first, reasons, next steps, risk

✎ DIAGRAM



Never say a fix before you've said the breakdown.

The fix-it script.

3. The breakdowns to memorise

✎ DIAGRAM

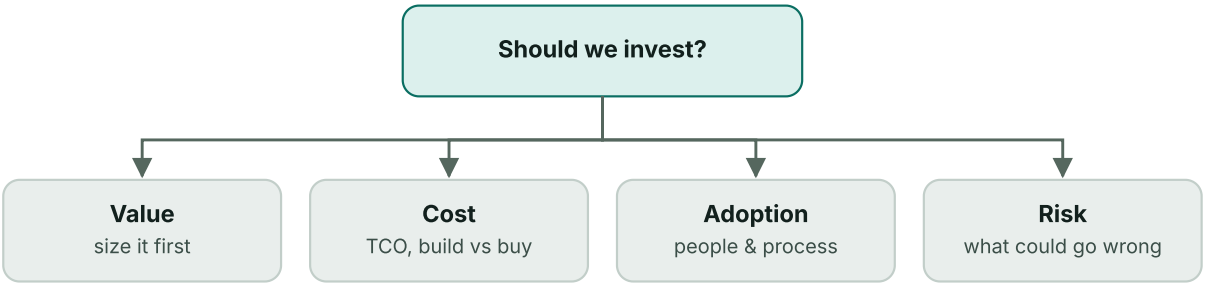
Problem	Break it into
Wrong answers	retrieval → generation
Too expensive	requests × tokens per request × price
Too slow	retrieval → reading → writing → streaming
Unsafe	inputs → tools → outputs
Is it good?	golden set → metrics → bar → CI → monitoring

Step 2 of the fix-it script for each kind of problem.

4. Business case

- Branches: **value** (first) → **cost** (build vs buy, full TCO) → **adoption** → **risk**
- Size value from the client's numbers; **challenge vendor claims** with reference customers
- Ask 'is that saving real?': time saved only becomes money if headcount changes or capacity is used (more patients, more leads)
- **Test the assumption**: 'what if it's 15% instead of 20%?'
- If it's marginal, **pilot where the value concentrates** (e.g. the 50 stores with 60% of the waste)
- Recommend: **answer + number + pilot + success bar + main risk**

✎ DIAGRAM



Start with value: if the prize is small, nothing else matters.

5. Tech maths

- **TCO** = buy/build + implement + train + run · **ROI** = (benefit − cost) ÷ cost · **Payback** = upfront ÷ annual savings
- Cost per message = monthly team cost ÷ messages (e.g. $75k ÷ 50k = $1.50)

6. Design it

- Clarify → requirements with numbers → **simplest working design** → where it breaks → evals and safety → trade-offs

7. Explain it (60 seconds)

- Define → mechanism → when (and when not) → trade-off → example
- Compare: shared goal → three differences → when each → your default

SAY IT

"Yes, but start with a pilot in X. It pays back in N months even at the low case. Success means Y; the main risk is Z, which we'd manage by W."